M165 - NOSQL-DATENBANKEN EINSETZEN

EINSTIEG

Datenbanken verfolgen das Ziel, grosse Datenmengen persistent und integer zu speichern. Dabei sollen Speicher- und Lesevorgänge schnell durchgeführt werden können.

INSTALLATION

Installiere die folgenden Softwares für alle Users.

> *In dieser Dokumentation wird mit der NoSQL Datenbank **MongoDB** gearbeitet.*

Damit die folgenden Softwares unter dem Betriebssystem automatisch eingerichtet werden, empfiehlt sich...

- ...für **Linux** das Package **server**.
- ...für **Windows** das Package **msi**.

MongoDB Community Server

Auf der Webseite https://www.mongodb.com/try/download/community kann die Software für das bevorzugte Betriebssystem heruntergeladen werden.\

MongoDB-Shell

Auf der Webseite https://www.mongodb.com/try/download/shell kann die Software für das bevorzugte Betriebssystem heruntergeladen werden.

Test

Folgend kann sichergestellt werden, dass der MongoDB Server läuft:

Bash (Linux) **PowerShell (Windows)**

```
get-service mongodb
```

Mit der Shell-Software kann eine Verbindung zum Server hergestellt werden:

Terminal

```
mongosh
```

CLOUD

An Stelle einer lokalen Datenbank, können Datenbanken auch in Clouds betrieben werden.

Datenbank erstellen

Folge dieser Anleitung, um eine kostenlose Datenbank in der Cloud zu erstellen.

1. Registriere dich auf mongodb.com und logge dich anschliessend ein.
2. Erstelle einen Cluster: Die kostenlose Variante **Shared** genügt für die folgenden Übungen.

3. Wähle für die Datenbank einen beliebigen Namen (Standardmässig *Cluster0*).
4. Unter dem Menüpunkt *Database* ist die Datenbank unter dem gewählten Namen sichtbar.

Zugriff

1. Erstelle unter dem Menüpunkt *SECURITY->Database Access* einen neuen User mit Passwort.
2. Füge unter dem Menüpunkt *SECURITY->Network Access* deine aktuelle IP-Adresse hinzu.

Info

Wenn sich deine IP-Adresse ändert, muss diese Einstellung aktualisiert werden.
Alternativ kann der Zugriff von jeder IP-Adresse erlaubt werden.

Verbindung via Shell

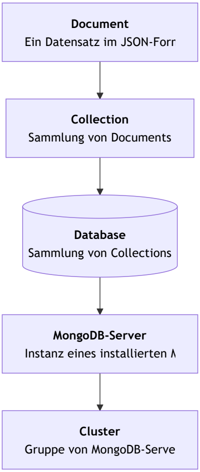
Über die Shell kann eine Verbindung zur Datenbank in der Cloud hergestellt werden.

1. Navigiere in der Cloud-Oberfläche zum Menüpunkt *Database*.
2. Klicke bei deiner Datenbank auf den Button *Connect*.
3. Wähle die Methode *Connect with the MongoDB Shell*.
4. Kopiere den Connection-String in deine Shell und führe den Befehl aus.

> *Passe optional den Usernamen durch den unter Database Access erstellten User an.*

5. Nach der Eingabe des Passworts, bist du mit der Datenbank verbunden.

KONZEPT



AUFGABEN

1. Installation

1. Installiere die Software **MongoDB Community Server**.
2. Installiere die Software **MongoDB-Shell**.
3. Stelle sicher, dass die Software **MongoDB Community Server** korrekt installiert ist und läuft.
4. Stelle sicher, dass die Software **MongoDB-Shell** korrekt installiert ist.

2. JSON

1. Beschreibe, wie Daten im JSON-Format dargestellt werden. Verwende in deiner Beschreibung die Begriffe *Key* und *Value*.
2. Wie sähe ein JSON aus, welches deinen *Namen*, *Vornamen* und *Alter* beinhaltet?
3. Ein Objekt *Adresse* beinhaltet Informationen zu *Strasse*, *Postleitzahl* und *Ort*. Wie sieht das JSON aus der vorhergehenden Aufgabe aus, würde das Objekt *Adresse* im JSON ergänzt werden?
4. Es soll eine Liste ergänzt werden, welche die Kinder der Person enthält.

3. Konzept

1. In welchem Format werden Documents dargestellt?
2. In welchem Format werden Documents in MongoDB gespeichert?
3. Was versteht man unter einer Collection?
4. Kann ein MongoDB-Server mehrere Datenbanken beinhalten?
5. Wozu werden *Cluster* im Zusammenhang mit der MongoDB genutzt?

4. Cloud

1. Erstelle eine DB in der Cloud.
2. Stelle sicher, dass via Shell auf die Datenbank in der Cloud zugegriffen werden kann.
3. Erstelle eine neue Datenbank *music* mit den Collections *songs* und *gigs*.
   Ergänze die Collections mit zwei Datensätzen und beliebigem Dateninhalt.
4. Erstelle einen DB-User, welcher ausschliesslich auf die Collection *songs* das Recht zum Lesen hat.
   Prüfe das Login, indem du dich mit der Cloud verbindest. Mit dem Befehl `show dbs` sollte dir ausschliesslich die Datenbank *music* angezeigt werden.
5. Entferne den Netzwerkzugang für deine IP-Adresse. Prüfe, ob der Zugang geblockt wird.
6. Richte einen Netzwerkzugang ein, welcher alle IP-Adressen akzeptiert. Prüfe erneut, ob er Zugang wieder funktioniert.

5. Zusatzaufgabe

1. Auf welchem Port läuft der MongoDB-Server?
2. Starte einen MongoDB-Server in einem Container. Der MongoDB-Server soll auf dem Port 55555 laufen.
   Wie lautet der dafür erforderliche Befehl?
3. Via Shell soll eine Verbindung mit dem MongoDB-Server im Container hergestellt werden.
   Wie lautet der dafür erforderliche Befehl?

BACKUP

Ein Backup einer Datenbank dient zur Sicherung der Daten. So können korrupte oder manipulierte Datensätze wieder hergestellt werden.

TOOLS

MongoDB Database Tools

Auf der Webseite https://www.mongodb.com/try/download/database-tools kann die Software **MongoDB Database Tools** für das bevorzugte Betriebssystem heruntergeladen werden.

IMPORT

Untenstehend ist der Befehl für den Import einer kompletten Datenbank beschrieben. Es wird eine Datenbank wieder hergestellt, welche ursprünglich mit `mongodump` exportiert wurde. Dem untenstehenden Befehl müssen drei Variablen ersetzt werden:

- `<URI>`: Connection-String (Optional: Ohne Angabe wird auf die lokale Installation zugegriffen)
- `<USER>`: Username (Optional: Wird benötigt, wenn der Datenbankzugriff nur durch einen User möglich ist, Bsp. in der Cloud)
- `<DBNAME>`: Name der Datenbank
- `<INPUT>`: Name des Datenbankexports

Terminal

```
mongorestore --uri="<URI>" --db=<DBNAME> --usertitle=<USER> <INPUT>
```

EXPORT

Untenstehend ist der Befehl für einen Export einer kompletten Datenbank beschrieben. Dem untenstehenden Befehl müssen drei Variablen ersetzt werden:

- `<URI>`: Connection-String (Optional: Ohne Angabe wird auf die lokale Installation zugegriffen)
- `<USER>`: Username (Optional: Wird benötigt, wenn der Datenbankzugriff nur durch einen User möglich ist, Bsp. in der Cloud)
- `<DBNAME>`: Name der Datenbank
- `<OUTPUT>`: Bezeichnung für den Datenbankexport

Terminal

```
mongodump --uri="<URI>" --db=<DBNAME> --usertitle=<USER> --out=<OUTPUT>
```

Es wird ein Verzeichnis mit dem Namen der Variable `<OUTPUT>` erzeugt. Im Verzeichnis ist eine Kopie der Datenbank, welche via `mongorestore` wieder hergestellt werden kann.

AUFGABEN

---

1. Grundlagen

1. Weshalb ist es wichtig, Backups von Datenbanken anzufertigen?
2. Mit welchem Werkzeug werden Backups in MongoDB angefertigt?

2. Import / Export

1. Hier befindet sich der Export einer Datenbank mit Restaurants: Entpacke und importiere die Daten in den lokalen Datenbankserver.

   Export

   - _index.files/db_restaurants.zip (3 MB)

2. Exportiere die Datenbank, welche in der vorherigen Aufgabe importiert wurde.
3. Importiere den Datenbank Export in die Datenbank in der Cloud.
4. Exportiere die Datenbank, welche in der voherigen Aufgabe importiert wurde.

3. Automatisches Backup

1. Erstelle ein Skript, welches beim Ausführen ein Backup der Datenbank erstellt. Das Backup soll im Verzeichnis *db_backup* abgespeichert werden. Der Name des Backups soll diesem Muster entsprechen: **YYYY_MM_DD**.
2. Stelle sicher, dass das Skript aus der vorhergehenden Aufgabe täglich um 02:00 Uhr ausgeführt wird. Dokumentiere die dazu notwendigen Einstellungen.
3. Das Skript soll nur 10 Backups behalten. Sobald mehr Backups vorhanden sind, werden die ältesten Backups gelöscht.

SHELL

Datenbankserver haben die Möglichkeit, mehrere Datenbanken zu managen. Daten können in den Datenbanken gespeichert, gelesen, gesucht, geändert und gelöscht werden.

Dieses Kapitel behandelt die Handhabung der Shell und wie mit der Datenbank gearbeitet werden kann.

Im Terminal kann mit dem Befehl `mongosh` die Mongoshell gestartet werden. Standardmässig verbindet sich die Shell mit der lokalen Datenbank.
Soll eine andere Datenbank erreicht werden, kann dem Befehl der Connection-String mitgegeben werden (Beispiel: `mongosh "mongodb://localhost:27017/"`).

DATENBANK

> *Ein Datenbankserver kann mehrere Datenbanken managen.*

Aktuelle Datenbank

Um die aktuell verwendete Datenbank anzuzeigen, wird der Befehl `db` eingegeben.

mongosh

```
test> db
test
```

Die Datenbank *test* ist standardmässig vorhanden und angewählt. Die Shell zeigt die angewählte Datenbank auf der Befehlszeile an.

Alle Datenbanken

Mit dem Befehl `show dbs` werden alle verfügbaren Datenbanken angezeigt.

mongosh

```
test> show dbs
admin   40.00 KiB
config  36.00 KiB
local   72.00 KiB
```

MongoDB hat nach der Installation bereits drei Datenbanken. Diese beinhalten Informationen und Einstellungen für die Datenbank selber.

Die Datenbank *test* wird angezeigt, sobald sie Daten enthält.

Datenbank wechseln

Um eine Datenbank zu wechseln oder neu zu erstellen, kann mit dem Befehl `use` und folgend dem Namen der Datenbank jene angewählt werden.

mongosh

```
test> use shop
switched to db shop
shop>
```

Auch eine neue Datenbank wird erst dann angelegt, wenn die ersten Documents darin gespeichert werden.

COLLECTIONS

Die in einer Datenbank enthaltenen Collections können angezeigt oder bearbeitet werden.

Collections anzeigen

mongosh

Input

```
db.getCollectionNames()
```

Output

```
[]
```

Die Ausgabe ist leer, wenn keine Collections vorhanden sind.

Collection erstellen

Eine neue Collection mit dem Namen *thing* wird erstellt.

mongosh

Input

```
db.createCollection("thing")
```

Output

```
{ ok: 1 }
```

Collection umbenennen

Eine bestehenden Collection *thing* soll in *item* umbenannt werden.

mongosh

Input

```
db.thing.renameCollection("item")
```

Output

```
{ ok: 1 }
```

Collection löschen

Die Collection *item* soll gelöscht werden.

mongosh

Input

```
db.item.drop()
```

Output

```
true
```

CRUD-OPERATIONEN

CRUD beinhaltet diese Operationen:

- *Create*
- *Read*
- *Update*
- *Delete*

Der Befehlssyntax entspricht diesem Muster:
`db.<collection>.<command>`

- `<collection>`: Die Collection für die der nachfolgende Command ausgeführt werden soll.
- `<command>`: Entspricht dem auszuführenden Befehl. Im Folgenden sind einige Befehle beschrieben.

CREATE

Ein Document kommt in eine Collection. Die Collection existiert zu diesem Zeitpunkt noch nicht und wird erst dann erstellt, wenn das erste Document gespeichert wird.

insertOne()

Der Befehl `insertOne()` fügt ein einzelnes Document hinzu.

Beispiel

In der Collection **item** soll ein neuer Artikel hinzugefügt werden.

Artikel:

```
{
    "name" : "Tastatur",
    "price": 15
}
```

mongosh

Input

```
db.item.insertOne(
  {
    "name": "Tastatur",
    "price": 15
  }
)
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("6299fd65e1b09a6cfc943c51")
}
```

insertMany()

Mehrere Documents können mit dem Befehl `insertMany()` hinzugefügt werden.

Beispiel

In der Collection **item** sollen drei neue Artikel hinzugefügt werden.

Artikel:

```
[
  {
    "name" : "Monitor",
    "price": 150
  },
  {
    "name" : "Maus",
    "price": 10
  },
  {
    "name" : "Lautsprecher",
    "price": 40
  }
]
```

mongosh

Input

```
db.item.insertMany(
  [
    {
      "name" : "Monitor",
      "price": 150
    },
    {
      "name" : "Maus",
      "price": 10
    },
    {
      "name" : "Lautsprecher",
      "price": 40
    }
  ]
)
```

Output

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("629f015f47ee1a787690d989"),
    '1': ObjectId("629f015f47ee1a787690d98a"),
    '2': ObjectId("629f015f47ee1a787690d98b")
  }
}
```

READ

Documents werden innerhalb einer Collection gelesen.

find()

Der Befehl `find()` sucht Documents in einer Collection. Der Befehl listet alle Documents in einer Collection.

Beispiel

Alle Documents in der Collection **item** sollen angezeigt werden.

mongosh

Input

```
db.item.find()
```

Output

```
[
  {
    _id: ObjectId("6299fd65e1b09a6cfc943c51"),
    name: 'Tastatur',
    price: 15
  },
  {
    _id: ObjectId("629f015f47ee1a787690d989"),
    name: 'Monitor',
    price: 150
  },
  {
    _id: ObjectId("629f015f47ee1a787690d98a"),
    name: 'Maus',
    price: 10
  },
  {
    _id: ObjectId("629f015f47ee1a787690d98b"),
    name: 'Lautsprecher',
    price: 40
  }
]
```

Mit Queries kann die Suche, mit Hilfe eines Filters, eingegrenzt werden.

Dem Befehl `find()` kann ein Query in den Klammer mitgegeben werden.
Die grundlegendsten Query-Befehle sind nachfolgend erklärt. Weitere Befehle können in der Dokumentation Dokumentation zur MongoDB gefunden werden.

ATTRIBUTE

Das Query sucht nach Documents, deren Attribute dem gesuchten entspricht.

Beispiel

Documents mit dem Namen *Tastatur* sollen angezeigt werden.

mongosh

Input

```
db.item.find( { "name": "Tastatur" } )
```

Output

```
[
  {
    _id: ObjectId("6299fd65e1b09a6cfc943c51"),
    name: 'Tastatur',
    price: 15
  }
]
```

VERGLEICHSOPERATOREN

Durch eine Abfrage mit Vergleichsoperatoren können Elemente ausgegeben werden, deren Attributwerte einer bestimmten Regel entsprechen.

Der Befehlssyntax ist wie folgt aufgebaut:
`{ <attribut>: { <operator>: <value> } }`

- `<attribut>`: Das Attribut welches verglichen werden soll.
- `<operator>`: Der Vergleichsoperator.
- `<value>`: Der durch den Vergleichsoperator zu vergleichende Wert.

Gleich

`$eq` liefert Documents deren Attribut den entsprechenden Wert hat.

Beispiel

Documents deren Preis 40 entspricht, sollen angezeigt werden.

mongosh

Input

```
db.item.find( { price: { $eq: 40 } } )
```

Output

```
[
  {
    _id: ObjectId("629f015f47ee1a787690d98b"),
    name: 'Lautsprecher',
    price: 40
  }
]
```

Ungleich

`$ne` liefert Documents deren Attribut nicht dem Wert entsprechen.

Beispiel

Documents deren Preis nicht 40 ist, sollen angezeigt werden.

mongosh

Input

```
db.item.find( { price: { $ne: 40 } } )
```

Output

```
[
  {
    _id: ObjectId("6299fd65e1b09a6cfc943c51"),
    name: 'Tastatur',
    price: 15
  },
  {
    _id: ObjectId("629f015f47ee1a787690d989"),
    name: 'Monitor',
    price: 150
  },
  {
    _id: ObjectId("629f015f47ee1a787690d98a"),
    name: 'Maus',
    price: 10
  }
]
```

Grösser

`$gt` liefert Documents deren Attribut den grösseren Wert hat.

Beispiel

Documents deren Preis grösser als 40 ist, sollen angezeigt werden.

mongosh

Input

```
db.item.find( { price: { $gt: 40 } } )
```

Output

```
[
  {
    _id: ObjectId("629f015f47ee1a787690d989"),
    name: 'Monitor',
    price: 150
  }
]
```

Grösser gleich

`$gte` liefert Documents deren Attribut den grösseren oder gleichen Wert hat.

Beispiel

Documents deren Preis grösser oder gleich 40 ist, sollen angezeigt werden.

mongosh

Input

```
db.item.find( { price: { $gte: 40 } } )
```

Output

```
[
  {
    _id: ObjectId("629f015f47ee1a787690d989"),
    name: 'Monitor',
    price: 150
  },
  {
    _id: ObjectId("629f015f47ee1a787690d98b"),
    name: 'Lautsprecher',
    price: 40
  }
]
```

Kleiner

`$lt` liefert Documents deren Attribut den kleineren Wert hat.

Beispiel

Documents deren Preis kleiner als 15 ist, sollen angezeigt werden.

mongosh

Input

```
db.item.find( { price: { $lt: 15 } } )
```

Output

```
[
  {
    _id: ObjectId("629f015f47ee1a787690d98a"),
    name: 'Maus',
    price: 10
  }
]
```

Kleiner gleich

`$lte` liefert Documents deren Attribut den kleineren oder gleichen Wert hat.

Beispiel

Documents deren Preis kleiner oder gleich 15 ist, sollen angezeigt werden.

mongosh

Input

```
db.item.find( { price: { $lte: 15 } } )
```

Output

```
[
  {
    _id: ObjectId("6299fd65e1b09a6cfc943c51"),
    name: 'Tastatur',
    price: 15
  },
  {
    _id: ObjectId("629f015f47ee1a787690d98a"),
    name: 'Maus',
    price: 10
  }
]
```

LOGICAL

Bedingungen können im Query aneinander geknüpft werden. Diese logischen Operationen stehen zur Verfügung:

- `$and`: Und-Verknüpfung
- `$or`: Oder-Verknüpfung
- `$nor`: Nicht-Oder-Verknüpfung
- `$not`: Invertierung

Beispiel

Documents, deren Preis zwischen 10 und 100 liegt, sollen angezeigt werden.

mongosh

Input

```
db.item.find( { $and: [{ "price": {$gt: 10}}, { "price": {$lt: 100} } ]} )
```

Output

```
[
  {
    _id: ObjectId("6299fd65e1b09a6cfc943c51"),
    name: 'Tastatur',
    price: 15
  },
  {
    _id: ObjectId("629f015f47ee1a787690d98b"),
    name: 'Lautsprecher',
    price: 40
  }
]
```

DARSTELLUNG

Die Ausgabe einer Abfrage kann nach belieben dargestellt werden. Dazu wir dem Befehl `find` ein zusätzlicher Parameter mitgegeben.

Mit Ausnahme der `_id` müssen Attribute, welche angezeigt werden sollen, speziell gewählt werden.
Soll die `_id` nicht angezeigt werden, muss diese speziell abgewählt werden.

Beispiel

Es soll ausschliesslich der Name aller Produkte (ohne `_id`) angezeigt werden.

mongosh

Input

```
db.item.find( {}, { "name": true, "_id": false } )
```

Output

```
[
  {
    name: 'Tastatur'
  },
  {
    name: 'Monitor'
  },
  {
    name: 'Maus'
  },
  {
    name: 'Lautsprecher'
  }
]
```

UPDATE

Um die Daten eines Documents zu verändern, muss das Document angesprochen werden und mit neuen Daten beliefert werden.

Ein Document kann mit denselben Methoden angesprochen werden, wie sie unter dem Kapitel Read erklärt sind.

Der Befehlssyntax ist wie folgt aufgebaut:
`db.collection.updateOne( <query>, <update> )`

- `<query>`: Query um ein Document zu selektionieren.
- `<update>`: Das neue Document.

updateOne()

Der Befehl `updateOne()` ändert ein Document.

Beispiel

Der Preis der Tastatur soll neu 20.- betragen.

mongosh

Input

```
db.item.updateOne( { "name": "Tastatur" }, { $set: { "price": 20 } } )
```

Output

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

> *Falls es mehrere Documents mit dem Attribut Tastatur gibt, wird nur das Document angepasst, welches als erstes gefunden wird.*
> *Für eine eindeutige Selektion empfiehlt sich, Documents mit ihrer ID anzusprechen.*

updateMany()

Der Befehl `updateMany()` ändert mehrere Documents.

Beispiel

Für alle Documents soll das neue Attribut *stock* mit der Stückzahl 0 ergänzt werden.
Damit alle Documents angepasst werden, wird der Filter leer gelassen.

mongosh

Input

```
db.item.updateMany( { }, { $set: { "stock": 0 } } )
```

Output

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 4,
  modifiedCount: 4,
  upsertedCount: 0
}
```

DELETE

Um Documents zu löschen, muss das Document angesprochen werden.

Ein Document kann mit denselben Methoden angesprochen werden, wie sie unter dem Kapitel Read erklärt sind.

Der Befehlssyntax ist wie folgt aufgebaut:
`db.collection.deleteOne( <query> }`

- `<query>`: Query um ein Document zu selektionieren.

deleteOne()

Der Befehl `deleteOne()` löscht ein einzelnes Document.

Beispiel

Der Preis der Tastatur soll neu 20.- betragen.

mongosh

Input

```
db.item.deleteOne( { "name": "Tastatur" } )
```

Output

```
{ acknowledged: true, deletedCount: 0 }
```

> *Falls es mehrere Documents mit dem Attribut Tastatur gibt, wird nur das Document gelöscht, welcher als erstes gefunden wird.*
> *Für eine eindeutige Selektion empfiehlt sich, Documents mit ihrer ID anzusprechen.*

deleteMany()

Der Befehl `deleteMany()` löscht mehrere Documents.

Beispiel

Alle Artikel, deren Preis tiefer als 50.- ist, sollen gelöscht werden.

mongosh

Input

```
db.item.deleteMany( { "price": { $lt: 50 } } )
```

Output

```
{ acknowledged: true, deletedCount: 2 }
```

DATENTYPEN

STRING

Beispiel

In der Collection **stringDT** wird ein Document hinzugefügt.

Document:

```
{
    "name" : "Tom Walter",
    "comment": "Name of a person"
}
```

mongosh

Input

```
db.stringDT.insertOne(
  {
    "name": "Tom Walter",
    "comment": "Name of a person"
  }
)
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("6349285e1a1a38ffe7a5e36b")
}
```

NUMBER

Folgende Datentypen für Zahlen sind vorhanden:

Datentyp	Beschreibung
int	32-bit integer
long	64-bit integer
double	64-bit floating point
decimal	128-bit floating point (IEEE 754)

Beispiel

In der Collection **numberDT** sollen vier Documents eingefügt werden. Dabei wird jeweils eine Zahl inklusive Beschreibung in unterschiedlichen Datentypen abgespeichert.

mongosh

Input

```
db.numberDT.insertMany(
  [
    {
      "number": 165,
      "comment": "Datatype int"
    },
    {
      "number": NumberLong("166"),
      "comment": "Datatype long"
    },
    {
      "number": 167.5,
      "comment": "Datatype double"
    },
    {
      "number": NumberDecimal("168.5"),
      "comment": "Datatype decimal"
    }
  ]
)
```

Output

```
{
  acknowledged: true,
  insertedIds:
   {
     '0': ObjectId("63492e291a1a38ffe7a5e378"),
     '1': ObjectId("63492e291a1a38ffe7a5e379"),
     '2': ObjectId("63492e291a1a38ffe7a5e37a"),
     '3': ObjectId("63492e291a1a38ffe7a5e37b")
   }
}
```

BOOLEAN

Beispiel

In der Collection **boolDT** wird ein Document hinzugefügt.

Document:

```
{
    "isMongoDBHard" : false,
    "amIEnjoying": true
}
```

mongosh

Input

```
db.boolDT.insertOne(
  {
    isMongoDBHard: false,
    amIEnjoying: true
  }
)
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("634930be1a1a38ffe7a5e37e")
}
```

DATE

Beispiel

In der Collection **dateDT** wird auf vier Arten ein Datum eingefügt.

mongosh

Input

```
db.dateDT.insertMany(
  [
    {
      date: Date(), comment: "current date"
    },
    {
      date: new Date(), comment: "current date as ISO Date"
    },
    {
      date: ISODate(), comment: "current date as ISO Date"
    },
    {
      date: ISODate("2012-12-19T06:01:17.171Z"), comment: "specific date as ISO Date"
    }
  ]
)
```

Output

```
{
  acknowledged: true,
  insertedIds:
   {
     '0': ObjectId("6349324e1a1a38ffe7a5e37f"),
     '1': ObjectId("6349324e1a1a38ffe7a5e380"),
     '2': ObjectId("6349324e1a1a38ffe7a5e381"),
     '3': ObjectId("6349324e1a1a38ffe7a5e382")
   }
}
```

Zeitzonen

Wird das Datum im ISO-Format angegeben, hat es diesen Aufbau `2012-12-19T06:01:17.171Z`. Das `Z` steht für **Zulu Time**, ein anderes Wort für **UTC**.
MongoDB speichert die Zeit immer im UTC-Format.

Beispiel

In der Winterzeit (UTC+01:00) wird der Zeitpunkt **11.11.2000, 11:11 Uhr** dargestellt:
`2000-11-11T11:11+01:00`

In der Sommerzeit (UTC+02:00) wird der Zeitpunkt **01.08.2000, 21:00 Uhr** dargestellt:
`2000-08-01T21:00+02:00`

OBJECT

Über den Datentyp Object kann ein Document in einem Document gespeichert werden.

Beispiel

In der Collection **objectDT** wird ein Document hinzugefügt.

Document:

```
{
    "name" : "Hotel Sea",
    "host": {
      "host_id": "5448114",
      "host_name": "David"
    }
}
```

mongosh

Input

```
db.objectDT.insertOne(
  {
      "name" : "Hotel Sea",
      "host": {
        "host_id": "5448114",
        "host_name": "David"
      }
  }
)
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("6349341c1a1a38ffe7a5e383")
}
```

ARRAY

Beispiel

In der Collection **arrayDT** wird ein Document hinzugefügt.

Document:

```
{
    "pass": ["Albula","Furka","Gotthard","Grimsel","Nufenen","Susten","Oberalp"],
    "country": "Switzerland"
}
```

mongosh

Input

```
db.arrayDT.insertOne(
  {
      "pass": ["Albula","Furka","Gotthard","Grimsel","Nufenen","Susten","Oberalp"],
      "country": "Switzerland"
  }
)
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("634934f61a1a38ffe7a5e384")
}
```

NULL

Null wird in Situationen vergeben, in denen das Attribut im Document zwar erscheinen soll, aber keinen Wert aufweist.

Beispiel

In der Collection **nullDT** wird ein Document hinzugefügt.

Document:

```
{
    "pass": null,
    "country": "Danmark"
}
```

mongosh

Input

```
db.nullDT.insertOne(
  {
      "pass": null,
      "country": "Danmark"
  }
)
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("634938fc1a1a38ffe7a5e385")
}
```

BINARY

Binäre Daten (Bsp.: Dokumente, Musik-, Filmdateien oder anderen Dateiformaten) können mit dem Datentyp Binary gespeichert werden.

Beispiel

In der Collection **binaryDT** wird ein Document hinzugefügt.

Document:

```
{
    "name": "my_txt",
    "extension": "txt",
    "content": "VGhpcyBpcyBhIHNpbXBsZSB0ZXh0IGZpbGUu")
}
```

mongosh

Input

```
db.binaryDT.insertOne(
  {
      "name": "my_txt",
      "extension": "txt",
      "content": BinData(0, "VGhpcyBpcyBhIHNpbXBsZSB0ZXh0IGZpbGUu")
  }
)
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("634938fc1a1a38ffe7a5e385")
}
```

TIMESTAMP

Beispiel

In der Collection **timestampDT** werden die gespeicherten Daten mit einem Zeitstempel versehen.

mongosh

Input

```
db.timestampDT.insertMany(
  [
    {
      "no": "LX233",
      "arrival": true,
      "ts": Timestamp()
    },
    {
      "no": "PB544",
      "arrival": true,
      "ts": Timestamp()
    },
    {
      "no": "XW775",
      "arrival": true,
      "ts": Timestamp()
    }
  ]
)
```

Output

```
{
  acknowledged: true,
  insertedIds:
   {
     '0': ObjectId("634941cb1a1a38ffe7a5e388"),
     '1': ObjectId("634941cb1a1a38ffe7a5e389"),
     '2': ObjectId("634941cb1a1a38ffe7a5e38a")
   }
}
```

LIMITIERUNG

Maximale Dokumentgrösse

Ein Dokument mit zu vielen Informationen ist in vielerlei Hinsicht problematisch. Aus diesem Grund begrenzt MongoDB die Grösse jedes Dokuments in einer Sammlung auf **16 MB**. Die Grenze von 16 MB reicht aus, um die relevanten Informationen zu speichern. Eine Sammlung kann beliebig viele Dokumente enthalten. Es gibt keine Begrenzung für die Grösse einer Sammlung. Selbst wenn eine Sammlung den Speicherplatz des zugrunde liegenden Systems überschreitet, können Sie vertikale oder horizontale Skalierung nutzen, um die Kapazität der Sammlung zu erhöhen.

Begrenzung der Verschachtelungstiefe

Ein MongoDB BSON-Dokument unterstützt eine Verschachtelungstiefe von bis zu 100 Ebenen, was mehr als ausreichend ist. Verschachtelte Dokumente sind eine hervorragende Möglichkeit, Daten übersichtlich darzustellen. Sie liefern alle Informationen auf einen Blick und vermeiden mehrere Abfragen, um eine bestimmte Information zu erhalten.

Mit zunehmender Verschachtelungstiefe treten jedoch Probleme hinsichtlich der Leistung und des Speicherverbrauchs auf. Betrachten wir beispielsweise einen Treiber, der ein Dokument in eine Objektstruktur zerlegt. Während des Durchlaufs greift der Scanner, sobald ein neues Unterdokument gefunden wird, rekursiv auf die verschachtelten Objekte zu und verwaltet dabei einen Stapel bereits gelesener Informationen. Dies führt zu einer hohen Speicherauslastung und einer langsameren Leistung.

Durch die Festlegung einer Verschachtelungstiefe von maximal 100 Ebenen vermeidet MongoDB solche Probleme. Wenn sich eine derart tiefe Verschachtelung jedoch nicht vermeiden lässt, können Sie überlegen, die Sammlungen in zwei oder mehr Teile aufzuteilen und **Dokumentverweise** zu verwenden.

Regeln für Feldnamen

MongoDB hat einige Regeln für Dokumentfeldnamen, die im Folgenden aufgeführt sind:

1. Der Feldname darf kein Nullzeichen enthalten.
2. Nur Felder in einem Array oder einem eingebetteten Dokument dürfen einen Namen haben, der mit einem Dollarzeichen ($) beginnt. Bei Feldern auf oberster Ebene darf der Name nicht mit einem Dollarzeichen ($) beginnen.
3. Dokumente mit doppelten Feldnamen werden nicht unterstützt. Laut der MongoDB Dokumentation wird beim Einfügen eines Dokuments mit doppelten Feldnamen kein Fehler ausgegeben, das Dokument wird jedoch nicht eingefügt. Selbst die Treiber verwerfen die Dokumente stillschweigend. In der Mongo-Shell wird ein solches Dokument hingegen korrekt eingefügt. Das resultierende Dokument enthält jedoch nur das zweite Feld. Das bedeutet, dass das zweite Vorkommen des Feldes den Wert des ersten überschreibt.

AUFGABEN

---

1. Shell

1. Mit welchem Befehl können alle Datenbanken in der Shell angezeigt werden?
2. Mit welchem Befehl können alle Collections angezeigt werden?
3. Wie kann eine Datenbank erstellt werden?
4. Wie lautet die Befehlsfolge, um die Datenbank `cloud` mit der Collection `pictures` zu erstellen. In der Collection soll ein Document gespeichert werden, welches diese Attribute `name` und `url` hat.

2. Create

Es soll eine NoSQL-Datenbank mit Filmen erstellt werden. Der Film soll auf Basis der vorgegebenen Parameter im JSON-Format dargestellt werden.

2.1 Eigenes JSON-Dokument erstellen

1. Erstelle ein JSON-Dokument auf Basis des Films *Die Schöne und das Biest*. Anbei die Key/Value-Paare.

```
Id = 14253
Title = Beauty and the Beast
Release Year = 2016
Language = English
Genre = Romance
Director = Christophe Gans
Runtime = 112
```

2. Validiere das JSON Document (z.B. https://jsonlint.com/), ob es synthaktisch gültig ist.
3. Füge das Document in die Datenbank *media*, in die Collection *movies* und gib das Document im Client via mongosh aus.

2.2 Erstellen von verschachtelten Objekten

Die Inhalte der vorhergehenden Aufgabe sollen erweitert werden.

1. Erweitere den Film um ein Rating, welches eine Bewertung von 0-10 (Decimal) und die Anzahl Stimmen (Integer) beinhaltet.
2. Aktualisiere das Document aus Aufgabe 2.1 und gib das Document im Client via mongosh aus.

2.3 Verwenden von Array-Feldern

Den Filmen sollen Kommentare hinzugefügt werden. Dabei wird der vollständige Text des Kommentars zusammen mit Benutzerdetails wie Name, E-Mail und Datum angeben.

1. Erweitere den Film mit zwei Kommentaren:

```
// Comment #1
Name = Talisa Maegyr
Email = oona_chaplin@gameofthron.es
Text = Rem itaque ad sit rem voluptatibus. Ad fugiat...
Date = 2017-08-22T11:45:03.000+00:00
// Comment #2
Name = Melisandre
Email = carice_van_houten@gameofthron.es
Text = Perspiciatis non debitis magnam. Voluptate...
Date = 2020-06-22T07:31:47.000+00:00
```

2. Aktualisiere das Document aus Aufgabe 1.1 und gib das Document im Client via mongosh aus.

2.4 Ergänze Filme

Ergänze die Datenbank.

1. Füge der Datenbank zwei beliebige Filme hinzu.
2. Füge zu jedem Film zwei beliebige Kommentare hinzu.

3. Read

Die folgenden Aufgaben beziehen sich auf die Restaurant-Datenbank, welche in einem vorgängigen Kapitel importiert wurde.

Folgend ist jeweils der Befehl oder die Befehlsequenz gesucht, um das gewünschte Ergebnis zu erhalten.

1. Liste alle Restaurants mit italienischer Küche (**Italian**) auf.
2. Finde das Restaurant mit dem Namen **7B Bar**.
3. Finde alle Restaurants, welche den Begriff **Steakhouse** im Namen tragen.
4. Finde alle Restaurants, welche die Postleitzahl **11219** besitzen.
5. Es gibt ein Restaurant, welches in einer Bewertung mehr als 100 Punkte (score) erhalten hat. Mit welchem Befehl kann es gefunden werden?
6. Finde alle Restaurants, welche eine irische Küche haben (**Irish**) und sich in **Queens** befinden.
7. Finde alle Restaurants in **Brooklyn**, welche entweder Backwaren (**Bakery**) oder Burgers (**Hamburgers**) anbieten.
8. Es sollen ausschliesslich die Namen aller Restaurants ausgegeben werden.
9. Finde alle Restaurants, welche **Seafood** anbieten. Es soll der Name und die Adresse ausgegeben werden.
10. Die Anzahl an Restaurants in Manhattan soll ermittelt werden.
11. Finde alle Restaurants, welche **Seafood** anbieten und keine Bewertung unter 5 haben.
12. Stelle die Restaurants aus der vorherigen Aufgabe mit Name und Adresse dar. Alle anderen Informationen sollen nicht angezeigt werden.

4. Update

Die folgenden Aufgaben beziehen sich auf die Restaurant-Datenbank, welche im vorgängigen Kapitel importiert wurde.

Folgend ist jeweils der Befehl oder die Befehlsequenz gesucht, um das gewünschte Ergebnis zu erhalten.

4.1 Neuer Besitzer

1. Das Restaurant mit dem Namen **Bully'S Deli** hat den Besitzer gewechselt und soll neu **Herbies Snacks** heissen.
2. Das Restaurant bietet unter neuer Führung Fastfood an. Passe die Küche an.
3. Der neue Besitzer hat eine Chance auf einen Neuanfang: Ersetze die bisherigen Bewertungen durch ein leeres Array.

4.2 Erweitern

Für neue Features wird die Datenbank erweitert.

1. Füge für jedes Restaurant ein neues Feld für den Ländercode **countrycode** ein. Da sich bisher alle Restaurants in der USA befinden, wird der Ländercode **US** eingetragen.
2. Manche Restaurants möchten ihre Öffnungszeiten anzeigen. Dazu sollen in ein Array **hours** die Öffnungszeiten für eine Woche eingetragen werden, beginnend bei Montag.
   Die Öffnungszeiten werden als Objekt mit den Feldern **from** und **to** als String eingetragen.
   Ergänze diese Öffnungszeiten für das Restaurant **Glorious Food**:

Tag	From	To
Mo	07:00	22:00
Di	12:00	22:00
Mi	07:00	22:00
Do	07:00	22:00
Fr	07:00	23:30
Sa	10:00	23:30
So	14:00	21:00

3. Zusätzliche Informationen sollen ein noch besseres Bild des Restaurants geben. In einem neuen Objekt namens **misc** sollen die zusätzlichen Informationen gespeichert werden.
   Ergänze diese Informationen für das Restaurant **Glorious Food**:

Bezeichnung	Beschreibung	Datentyp	Wert
hasWifi	WLAN-Zugang	Boolean	true
parking	Anzahl Parkplätze	Integer	15
webpage	Webseite	String	gloriousfood.com
phone	Telefonnummer	String	+1 123-456-7890

4. Im Objekt **misc** wird der Zeitstempel der Eröffnung ergänzt. Im Feld **opening** wird für das Restaurant **Glorious Food** das Datum 1. August 2022, welches um 7:00 Uhr ortszeit geöffnet hat, eingetragen.
   Wichtig: Beachte die Zeitzone.

5. Vertiefung

Die Datenbank *media* soll um Songs ergänzt werden. Dabei soll der Song direkt in der Datenbank abgespeichert werden.

1. Erstelle eine neue Collection mit dem Namen *songs*.
2. Ein Song soll folgende Informationen enthalten:
   - Name
   - Interpret
   - Album
   - Genre
   - Erscheinungsjahr
   - Länge
   - Datum, wann das Document eingefügt wurde

   Wähle geeignete Datentypen und speichere einen Song in deiner Datenbank ab.
3. Finde heraus, wie eine Song-Datei (.mp3) in der Datenbank abgespeichert werden kann.
4. In einer zusätzlichen Collection *playlists* können Playlisten abgespeichert werden.
   1. Überlege dir alle notwendigen Attribute für Daten dieser Collection.
   2. Füge der Collection zwei Playlisten hinzu.
5. Wie lautet eine Datenbankabfrage, welche alle Interpreten (jeder Interpret nur 1 Mal) ausgeben?
6. Wie lautet eine Datenbankabfrage, welche denjenigen Interpreten mit den meisten Songs ausgibt?

SOFTWARE

Die Datenbank kann über Softwares angesprochen werden.

Python C#

1. Erstelle ein Python Projekt
2. Installiere das Package **pymongo**
3. Führe diesen Code aus (der Connection-String muss angepasst werden):

```
from pymongo import MongoClient

connection_string = "mongodb://localhost:27017/"
client = MongoClient(connection_string)

print(client.server_info())
```

Der Code ist korrekt, wenn Informationen über die Datenbank (beispielsweise die Version) ausgegeben werden.

> *Im Folgenden werden Codefragmente gezeigt, in welchen bestehende Objekte (client) weiterhin genutzt werden. Der Code soll ergänzt werden.*

DATENBANKEN

Die folgenden Beispiele funktionieren nur dann, wenn mittels Objekt `client` eine Verbindung zum Datenbank-Server hergestellt wurde.

Liste aller Datenbanken

Mit diesem Code werden alle existierenden Datenbanken ausgegeben.

Python C#

```
dblist = client.list_database_names()

for db in dblist:
        print(db)
```

Prüfe bestehende Datenbank

Der Name der Datenbank wird in der Liste der existierenden Datenbanken gesucht:

Python C#

```
dblist = client.list_database_names()

if "admin" in dblist:
    print("Database exists.")
else:
    print("Database does not exist.")
```

Datenbank erstellen / selektieren

Das Erstellen und Selektieren einer Datenbank unterscheidet sich synthaktisch nicht.

Python C#

```
db = client["inf"]
```

> *Sollte die Datenbank noch nicht existieren, wird diese erstellt, sobald Daten in ihr gespeichert werden.*

COLLECTIONS

Die folgenden Beispiele funktionieren nur dann, wenn mittels Objekt `db` eine Datenbank selektiert wurde.

Liste aller Collections

Alle Collections in einer Datenbank werden aufgelistet.

Python C#

```
collist = db.list_collection_names()

for col in collist:
    print(col)
```

Prüfe bestehende Collection

Der Name der Collection wird in der Liste der existierenden Collections gesucht:

Python C#

```
collist = db.list_collection_names()

if "myCollection" in collist:
    print("Collection exists.")
else:
    print("Collection does not exist.")
```

Collection erstellen / selektieren

Das Erstellen und Selektieren einer Collection unterscheidet sich synthaktisch nicht.

Python C#

```
col = db["module"]
```

> *Sollte die Collection noch nicht existieren, wird diese erstellt, sobald Daten in ihr gespeichert werden.*

Collection löschen

Python C#

```
col.drop()
```

Sobald eine Datenbank keine Collections mit Daten mehr enthält, wird die Datenbank entfernt.

DOCUMENTS

Die folgenden Beispiele funktionieren nur dann, wenn mittels Objekt `col` eine Collection selektiert wurde.

> *Die Methoden der Collection sind gleich benannt, wie die Befehle in der Shell. Es wird deshalb ein Beispiel erklärt. Weitere Möglichkeiten können den Befehlen aus der Shell entnommen werden (siehe CRUD-Operationen).*

Im Code kann direkt mit Key-Value Paaren gearbeitet werden. Diese sind in Python als ein Dictionary Objekt gespeichert.

Insert

Python C#

```
module = { "number": 165, "description": "NoSQL-Datenbanken einsetzen" }

col.insert_one(module)
```

Find

Python C#

```
module = col.find_one()

print(module)
```

Query

Die Methode `find` kann mit einem Query ergänzt werden. So werden nur noch Documents gefunden, welche dem Query entsprechen.

Python C#

```
query = { "number": 165 }

modules = col.find(query)

for module in modules:
        print(module)
```

Weiter kann das Query mit Vergleichsoperatoren ergänzt werden (siehe Vergleichsoperatoren).

Python C#

```
query = { "number": { "$gt": 100 } }

modules = col.find(query)

for module in modules:
        print(module)
```

Suche ID

Die ID wird in der Datenbank als Typ von **ObjectId** gespeichert. Soll im Code nach der ID gesucht werden, muss die sichtbare ID (bestehend aus Zahlen und Buchstaben) in ein ObjectId gewandelt werden.

Python C#

```
import bson

id = bson.ObjectId("644a6d1c6ad2ed9113095057")
query = { "_id": id }

module = col.find_one(query)

print(module)
```

Update

Die Update-Methoden benötigen zwei Parameter:

1. Query, welcher die Documents definiert, welche aktualisiert werden.
2. Das aktualisiert Document.

Python C#

```
query = { "number": 165 }
newvalues = { "$set": { "description": "NoSQL-Datenbanken verstehen" } }

col.update_one(query, newvalues)
```

Delete

Die Delete-Methoden benötigt einen Query, welcher die zu löschenden Documents definiert.

Python C#

```
query = { "number": 165 }

col.delete_one(query)
```

KLASSEN

In einer objektorientierten Sprache sind zusammenhängende Daten oft an ein Objekt gebunden. Da diese Daten auch in der Datenbank zusammenhängend sind, werden oft ganze Objekte in einer Datenbank gespeichert oder von einer Datenbank gelesen.

Arrays und andere Objekte können in einem Objekt enthalten sein. Das Objekt kann in der Datenbank gespeichert oder von der Datenbank gelesen werden.

In den folgenden Beispielen wird ein Objekt der Klasse `Room` in die Datenbank gespeichert und wieder von der Datenbank gelesen.

Es werden zwei Files erzeugt:

Python C#

```
├── main.py
└── room.py
```

Python C#

room.py

```
class Room:
    def __init__(self, name, seats, is_reservable, _id = None):
        if(_id is not None):
            self._id = _id
        self.name = name
        self.seats = seats
        self.is_reservable = is_reservable
```

Der Konstruktor der Klasse `Room` enthält einen optionalen Parameter mit dem Namen `_id`.
Sollte das Objekt in die Datenbank gespeichert werden, soll das Attribut `_id` nicht vorhanden sein: Die Datenbank wird es selbständig erstellen.
Sollte das Objekt von der Datenbank gelesen werden, enthält der gelesene Dictionary das Attribut `_id` und instanziert das Objekt mit der ObjectId.

Objekt in Datenbank speichern

Dieser Code zeigt auf, wie ein Objekt als Document abgespeichert werden kann.

Python C#

main.py

```
from pymongo import MongoClient
from room import Room

connection_string = "mongodb://localhost:27017/"
db_rooms = MongoClient(connection_string)["buildings"]["rooms"]

room = Room("Pilatus", 12, True)

db_rooms.insert_one(room.__dict__)
```

Die Methode `__dict__` liefert die Attribute des Objekts als Dictionary aus. Der Dictionary kann direkt in der Datenbank gespeichert werden.

Objekt von Datenbank laden

Um ein aus der Datenbank gelesenes Document wieder in ein Objekt zu konvertieren, kann dieser Code genutzt werden:

Python C#

main.py

```
from pymongo import MongoClient
from room import Room

connection_string = "mongodb://localhost:27017/"
db_rooms = MongoClient(connection_string)["buildings"]["rooms"]

room = Room(**db_rooms.find_one())
```

Die Sterne in der letzten Codezeile bezwecken, dass die im Dictionary enthaltenen Key-Value Paare dem Konstruktor der Klasse übergeben werden.

Optionale Attribute

Attribute wie die ID werden nur einmalig gesetzt, wobei die ID von der Datenbanksoftware vergeben wird. Andere Attribute müssen möglicherweise von der Software gesetzt werden, wie beispielsweise ein Zeitstempel, der den Zeitpunkt speichert, zu dem ein Document hinzugefügt wird.
Ähnlich wie bei der ID wird im Konstruktor geprüft, ob das zu instanzierende Objekt von der Datenbank geladen wurde oder gänzlich neu erstellt wird.

An dieser Stelle kann geprüft werden, ob dieses einmalig zu setzende Attribut bereits existiert.

Python C#

main.py post.py

```python
from pymongo import MongoClient
from post import Post

connection_string = "mongodb://localhost:27017/"
db_posts = MongoClient(connection_string)["forum"]["posts"]

post = Post("Welcome to the forum")

db_posts.insert_one(post.__dict__)
```

DATA ACCESS OBJECT

Als *Data Access Object* wird die Klasse bezeichnet, welche die Schnittstelle zur Datenbank bildet. Sie abstrahiert CRUD-Operationen in der Datenbank.

Anknüpfend zum vorgängigen Beispiel wird ein neues File erzeugt.

Python C#

```
├── dao_room.py
├── main.py
└── room.py
```

Innerhalb dieser Klasse werden Methoden erstellt, welche das Handling mit der Datenbank vereinfachen.
Im folgenden Beispiel werden nur die Methoden *create* und *read* erstellt.

Python C#

main.py dao_room.py room.py

```python
from room import Room
from dao_room import Dao_room

dao_room = Dao_room("mongodb://localhost:27017/")

# Create
room_create = Room("Pilatus", 12, True)
dao_room.create(room_create)

# Read
room_read = dao_room.read()
```

FILES

Files wie Bilder, Videos, Office Dokumente, usw. können in Datenbanken gespeichert werden. Der Dateiinhalt und die Metadaten werden in separaten Attributen gespeichert.

MongoDB bietet mit `GridFS` eine Möglichkeit, um Files auf einfache Weise abzuspeichern und abzurufen.
`GridFS` speichert die Files in zwei Collections:

- `fs.chunks`: Files werden in dieser Collection als Chunks aufgeteilt gespeichert.
- `fs.files`: Die Metadaten zur Datei werden hier gespeichert.

Dieses Beispiel zeigt auf, wie Files mit Hilfe von `GridFS` in MongoDB gespeichert und ausgelesen werden können.

Python C#

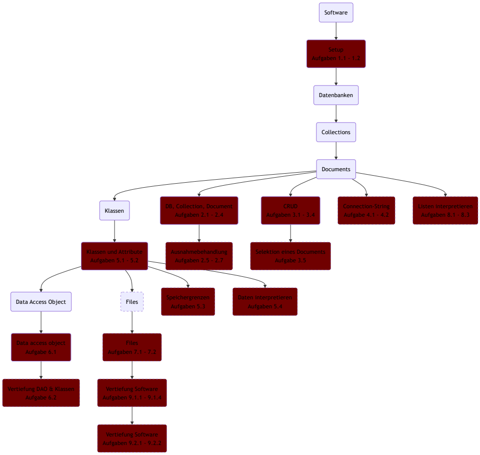
main.py

```python
from pymongo import MongoClient
import gridfs
import os

client = MongoClient('mongodb://localhost:27017/')
db = client['files']
fs = gridfs.GridFS(db)

path = input("File: ")

# Save
with open(path, 'rb') as file:
    file_id = fs.put(file, filetitle=os.path.basename(path))
print("File saved")


# Read
file = fs.get(file_id)
restore_path = os.path.join(".", file.filename)

data = fs.get(file_id).read()
with open(restore_path, 'wb') as file:
    file.write(data)
print("File restored")
```

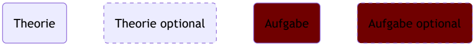
AUFGABEN

Leitfaden

Theorie und Aufgaben in diesem Kapitel können teilweise freiwillig verarbeitet werden. Die untenstehende Übersicht zeigt Abhängigkeiten zwischen Theorien und Aufgaben.



Legende



1. Setup

1. Erkläre die Abkürzung ODM und welche Funktion ODM wahrnimmt.
2. Richte dein System so ein, dass du über Python oder C# eine Verbindung zur Datenbank herstellen kannst. Erstelle dazu ein kleines Testprogramm.

2. DB, Collection, Document

Erstelle eine Applikation, die als Datenbank / Collection und Document Explorer funktioniert.

1. Wenn die Applikation gestartet wird, werden alle Datenbanken nach diesem Muster aufgelistet:

```
Databases
 - db1
 - db2
 - db3

Select Database:
```

2. Eine Datenbank kann selektiert werden, indem ihr Name eingegeben wird. Nach der Eingabe wird der Name der gewählten Datenbank und alle enthaltenen Collections angezeigt:

```
<db_name>

Collections
 - col1
 - col2
 - col3

Select Collection:
```

3. Eine Collection kann selektiert werden, indem ihr Name eingegeben wird. Nach der Eingabe wird der Name der Datenbank, der Collection und alle IDs der enthaltenen Documents angezeigt:

```
<db_name>.<col_name>

Documents
 - 63dd4f4
 - 63dd4f5
 - 63dd4f6
```

```
Select Document:
```

4. Ein Document kann selektiert werden, indem seine ID eigegeben wird. Nach der Eingabe wird der Name der Datenbank, der Collection und der Inhalt des gewählten Documents angezeigt:

   ```
   <db_name>.<col_name>.<id>

   content1: example
   content2: 605

   Press any button to return
   ```

5. Der Text **Press any button to return** wird in zwei Situationen angezeigt:

   - Der Inhalt eines Documents wird angezeigt.
   - Ein gewähltes Element hat keinen Inhalt (Bsp.: In einer Collection existiert kein Document).

   In beiden Fällen startet nach einem Tastendruck die Applikation von vorne.

6. Wenn kein anzuzeigendes Element (Datenbank, Collection, Document) vorhanden ist, wird dies angezeigt. Hier beispielsweise bei einer Datenbank:

   ```
   No Database
   ```

7. Sollte vom User etwas (Datenbank, Collection, Document) gewählt werden, was nicht vorhanden ist, wird eine Meldung ausgegeben. Der User kann nochmals wählen.

3. CRUD

Die folgenden Aufgaben beziehen sich auf die Restaurant-Datenbank, welche in einem vorgängigen Kapitel importiert wurde.

1. Die Restaurants befinden sich in Stadtbezirken. Alle Stadtbezirke sollen ausgegeben werden, jedoch nicht doppelt.

2. Es sollen die top 3 Restaurants ausgegeben, welche im Durchschnitt das höchste Rating (score) haben.

3. Finde das Restaurant, welches geografisch am nächsten zum Restaurant mit dem Namen **Le Perigord** liegt.

4. Erstelle eine Applikation, mit welcher ein Restaurant gesucht werden kann.

   - Es soll nach den Attributen **Name** und **Küche** gesucht werden können.
   - Die Suche der Attribute ist optional: Wird ein Attribut für die Suche leer gelassen, wird es ignoriert.
   - Ein teilweise übereinstimmender Suchbegriff (Bsp.: **Steakhouse**) liefert alle Restaurants, welche den Begriff beinhalten.

5. Ergänze die Applikation aus der vorhergehenden Aufgabe. Einem gesuchten Restaurant kann eine Bewertung vergeben werden.

   - Liefert die Suche mehrere Restaurants als Ergebnis, kann eines ausgewählt werden. Dabei soll die *ID* des Documents im Programm gespeichert werden.
   - Die Bewertung wird mit dem aktuellen Datum versehen und dem Document ergänzt.

4. Connection-String

Im Connection-String sind Zugangsdaten zur Datenbank enthalten. Der Connection-String soll aus sicherheitstechnischen Gründen nicht im Source Code, sondern in einer Umgebungsvariable gespeichert werden.

1. Recherchiere, wie in Python oder C# Umgebungsvariablen gelesen werden können.
   Erstelle ein Programm, welches exemplarisch den Inhalt der Umgebungsvariable **PATH** ausliest und ausgibt.

2. Speichere den Connection-String in einer Umgebungsvariable und stelle mit einer Testsoftware eine Verbindung zur Cloud Datenbank her.

5. Power Statistic

Mit der Python Library `psutil` kann die Auslastung von CPU und RAM angezeigt werden. Bei C# gibt es für diesen Zweck das NuGet-Paket `Hardware.Info`.

1. Erstelle eine Klasse mit dem Namen `Power`, welche diese Attribute speichert:

   - CPU (in Prozent)
   - RAM total
   - RAM in Verwendung
   - Zeitstempel

   Achte darauf, dass der Zeitstempel und die Messungen von CPU und RAM nur dann erstellt werden muss, wenn dem Konstruktor keine Werte übergeben werden.

2. Die aktuelle Auslastung von CPU und RAM (total + in Verwendung) werden im Sekundentakt in der Datenbank abspeichert.

3. Die Applikation soll selbständig dafür sorgen, dass nicht mehr als 10000 Logs vorhanden sind. Sobald mehr als 10000 Logs gespeichert sind, werden die ältesten Logs gelöscht, sodass nur noch 10000 Logs vorhanden sind.

4. Erstelle eine zusätzliche Applikation, welche die Daten in einem Graph anzeigt. Nutze dazu für Python die Library `matplotlib` und für C# das NuGet-Paket `OxyPlot.WindowsForms`.

6. DAO

1. Ergänze die Klasse `Dao_room` aus diesem Beispiel um die Methoden `update` und `delete`.

2. Erstelle eine Datenbank, in welcher Jokes gespeichert werden.

   1. Entwirf die Klasse mit diesen Attributen:

      - `text`: Text des Jokes.

    - `category`: Liste von Kategorien welcher der Joke zugehörig ist.
    - `author`: Verfasser des Jokes.

  2. Erstelle die DAO-Klasse mit diesen Methoden:

    - `insert`: Neuer Witz kann hinzugefügt werden
    - `get_category`: Die Kategorie kann als Parameter übergeben werden. Alle Witze dieser Kategorie werden zurückgegeben.
    - `delete`: Die ID des zu löschenden Jokes wird übergeben. Die Methode löscht den Joke.

7. Files

1. Nutze den Code aus dem Beispiel, um ein File zu speichern und wieder herzustellen.

    1. Beobachte die erstellten Collections in der Datenbank.
    2. Zu einem File werden mehrere Documents angelegt. Wie wird die Verbindung der einzelnen Documents zueinander hergestellt.
    3. In welcher Codierung werden die Rohdaten des Files gespeichert?

2. Erstelle eine Applikation, welches Bilder als Fotoalbum speichert. Jedes Bild enthält als Metadaten die Information, zu welchem Album das Bild gehört.

    1. Finde mit Hilfe der Dokumentation heraus, wie Metadaten mit `GridFS` zu einem File gespeichert werden können.
    2. Neue Fotos können einem beliebigen Fotoalbum hinzugefügt werden.
    3. Fotos eines Fotoalbums können aus der Datenbank heruntergeladen werden.

8. Listen interpretieren

Die folgenden Aufgaben beziehen sich auf die Restaurant-Datenbank, welche in einem vorgängigen Kapitel importiert wurde.

Die Restaurant-Datenbank verfügt über eine zweite Collection mit dem Namen **neighborhoods**. Die gespeicherten Documents enthalten Koordinaten, welche durch Verbindungen ein Polygon darstellen.

Es soll eine Applikation erstellt werden, welche die Polygone grafisch darstellt.

1. Mit Hilfe der Python Library **Pillow** kann ein Bild gezeichnet werden. Bei C# gibt es für diesen Zweck das NuGet-Paket `System.Drawing`. Analysiere dazu diesen Code:

Python C#

```
from PIL import Image, ImageDraw

im = Image.new(mode="RGB", size=(200, 200))

draw = ImageDraw.Draw(im)
draw.line((100, 200, 150, 180), fill=0x00ffff, width=3)

im.show()
```

2. Lies die in der Collection **neighborhoods** gespeicherten Koordinaten eines einzelnen Polygons ein und stelle es als Bild dar.
3. Lies die in der Collection **neighborhoods** gespeicherten Koordinaten aller Polygone ein und stelle sie als einzelnes Bild dar.

9. Jukebox

Mit Python oder C# soll eine Jukebox programmiert werden, welche Audiofiles speichert und abspielt. Die Jukebox besteht aus zwei Softwares:

- Management: Hinzufügen, Ändern und Löschen von gespeicherten Musikstücken
- Player: Suchen von Musikstücken, Stücke in einer Playlist hinzufügen, Spielen der Playlist

9.1 Management

Beim Hinzufügen eines Songs muss mindestens ein **Name** für das Stück und ein **Interpret** angegeben werden. Optional können **Album**, **Genre** und **Erscheinungsjahr** angegeben werden.

1. Erstelle eine Klasse für die Songs und wähle geeignete Attribute.

2. Programmiere das Feature, dass ein Song in der Datenbank gespeichert werden kann.

3. Ergänze das Feature, dass ein Song aus der Datenbank geändert werden kann. Dazu muss der Song gesucht und angewählt werden können.

4. Ergänze das Feature, dass ein Song aus der Datenbank gelöscht werden kann. Dazu muss der Song gesucht und angewählt werden können.

9.2 Player

1. Der Player bietet Suchfunktionen für Name, Interpret, Album und Genre an. Die Suchfunktion arbeitet nach diesem Algorithmus:

    1. Gross-/Kleinschreibung wird ignoriert.

    2. Songs werden auch dann aufgelistet, wenn nur ein Teil des Suchbegriffs übereinstimmt. Beispiel: Suche nach **the Wall** findet auch **Another Brick in the Wall**.

    3. Suchparameter können kombiniert werden. Beispiel: Suche nach Interpret **Beatles** und Name **Back** findet unter anderem **Get Back** und **I'll Be Back**.

2. Ein gefundener Song kann der Playlist hinzugefügt werden. Die Songs werden in der Reihe abgespielt, wie sie eingefügt wurden (*first in first out*).
Sollte sich kein Song in der Playliste befinden, wird ein zufälliger Song abgespielt.